

LOTUS VALLEY INTERNATIONAL SCHOOL,
GURUGRAM



ACADEMIC YEAR: 2025-26

PROJECT REPORT ON
SUTDENT MANAGEMENT SYSTEM

NAME: Anav Shah

CLASS: XII Tesla

SUBJECT: COMPUTER SCIENCE

SUBJECT CODE: 083

PROJECT GUIDE: Ms. JAGMEET KAUR

PGT (CS)

LOTUS VALLEY

INTERNATIONAL SCHOOL

GURUGRAM, DELHI

CERTIFICATE

This is to certify that Anav Shah of class XII Tesla has successfully completed the project work on Computer Science (083) on the topic Student Management for class XII Practical Examination of the Central Board of Secondary Education in the year 2025-26. It is further certified that this project is the original work of the individual.

(Subject Teacher)

(Principal)

(External examiner)

ACKNOWLEDGEMENT:

I take this opportunity to sincerely thank all those who supported and guided me throughout the successful completion of this project, undertaken as a part of my Class XII Computer Science curriculum.

First and foremost, I would like to express my heartfelt gratitude to **Mrs. Jagmeet Kaur**, my Computer Science teacher, for her constant guidance, encouragement, and valuable insights that helped me understand and implement the concepts involved in this project effectively.

I am also deeply thankful to our respected Director Principal, **Mrs. Anita Malhotra**, for providing a positive academic environment and for her motivational support, which served as a driving force throughout the development of this project.

A special note of thanks to my parents for their unwavering support, patience, and continuous motivation, which inspired me to give my best effort.

Last but not least, I would like to extend my sincere appreciation to my friends and classmates for their constructive feedback and suggestions, which played a significant role in refining the project.

Their collective support made this learning journey both enriching and memorable.

Anav Shah
XII Tesla

INDEX

S.NO.	TOPIC	PAGE NO.
1.	INTRODUCTION	5
2.	OBJECTIVES	6
3.	FUTURE SCOPE	7
4.	THEORETICAL BACKGROUND	9
5.	SOURCE CODE	11
6.	OUTPUT	19
7.	BIBLIOGRAPHY	21

INTRODUCTION:

A **Student Management System** in Python is a simple application designed to manage student-related data effectively. It is ideal for schools or institutions looking to automate student record handling such as registration, updates, marks entry, and performance reports.

Name of the Project:

Student Management System

OBJECTIVES:

The main objectives of this project are to:

- **Register Students:** Enter details such as name, roll number, class, gender, and contact information.
- **Update Student Records:** Modify student data like marks, contact info, etc.
- **Delete Records:** Remove students from the database if needed.
- **Search and Display Records:** Retrieve specific student records using roll number or name.
- **Generate Reports:** Display summaries such as class performance using graphs.

Modules Used:

- **Python**
- **MySQL Interface**

FUTURE SCOPE:

The current implementation of the Student Management System provides a basic and functional model using Python, MySQL, and File Handling. However, in the real world, such systems can be significantly enhanced for better usability, security, automation, and scalability. Below are the key areas where this project can be improved or extended in the future:

1. GUI Integration

- Implement a Graphical User Interface using Python's Tkinter. It Improves user experience, makes the system more intuitive and user-friendly, especially for school staff unfamiliar with CLI.

2. Authentication and Security

- Implement user authentication using usernames and passwords. Encrypt sensitive information using libraries and protects against unauthorized access.

3. Advanced Search and Filtering

- Enable multi-field search, sorting, and filtering (e.g., "find students with marks < 40", or "top 5 students by class"). It Saves time, increases data analysis capacity.

4. Integration with Cloud-Based Databases

- Implementing cloud technologies would enable remote access to patient information, allowing doctors and healthcare professionals to view and update records in real-time from any location.

5. Mobile App Version

- Future Upgrade: Develop an Android/iOS app using tools like Kivy, Flutter, or React Native.
- Benefits:
 - Students and teachers can use it on-the-go.
 - Real-time attendance/marks update through the app.

6. AI-Based Predictions and Alerts

- Future Upgrade:
 - Use machine learning to predict:
 - Students at risk of failing.
 - Future performance trends.
 - Alert teachers automatically.
- Benefits: Enables early intervention to help struggling students.

THEORETICAL BACKGROUND:

The theoretical foundation of the Student Management System project lies in key areas of Computer Science such as Programming in Python, Database Management using MySQL, and File Handling Techniques. Understanding these core concepts is essential for developing reliable, efficient, and scalable data management solutions.

PYTHON PROGRAMMING LANGUAGE

Python is an interpreted, object-oriented, high-level programming language with dynamic semantics. Created by **Guido van Rossum** in 1991, it is known for its readability and minimal syntax, which makes it ideal for beginners and professionals alike.

Key Features Relevant to This Project:

- **Easy Syntax:** Closer to English, easy to read and write.
- **Dynamic Typing:** No need to declare data types.
- **Rich Standard Library:** Includes built-in modules for file handling, database access, exception handling, etc.
- **Cross-platform:** Works on Windows, Mac, Linux.
- **Support for Modules:** Like OS, pickle, and mysql.connector used in this project.

MYSQL – RELATIONAL DATABASE MANAGEMENT SYSTEM

MySQL is an open-source **RDBMS (Relational Database Management System)** used for storing, managing, and retrieving structured data. It uses **SQL (Structured Query Language)** to perform database operations like insert, update, delete, and query.

Key Concepts:

- **Database:** A collection of related data stored in tables.
- **Table:** Structured format of rows and columns (like Excel).
- **Primary Key:** Uniquely identifies a record (e.g., Roll Number).
- **SQL Queries:** Instructions to perform tasks.

Example SQL Table:

```
CREATE TABLE Students (  
    RollNo INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Class VARCHAR(10),  
    Marks INT);
```

Basic SQL Commands Used:

- **INSERT INTO** Students VALUES (101, 'Raj', '12A', 88);
- **SELECT *** FROM Students;
- **DELETE FROM** Students WHERE RollNo = 105;
- **UPDATE** Students SET Marks = 95 WHERE RollNo = 101;

PYTHON-MYSQL CONNECTIVITY

Python connects to MySQL using the `mysql.connector` module.

Steps to connect Python with MySQL:

1. Install connector:

```
pip install mysql-connector-python
```

2. Establish connection:

```
import mysql.connector
```

```
con = mysql.connector.connect(host='localhost', user='root',  
passwordd='', database='school')
```

3. Execute query:

```
cursor = con.cursor()
```

```
cursor.execute("SELECT * FROM Students")
```

```
data = cursor.fetchall()
```

Advantages of using MySQL in this project:

- Structured and scalable data storage.
- Secure and fast access.
- Supports multiple users and connections.
- Easy to write and understand queries.

This detailed theoretical foundation provides clarity on the programming logic, modules, and technologies used in building the **Student Management System**, and reflects sound understanding of real-world application development.

SOURCE CODE

```
import mysql.connector

# Add a new student

def add_student(name, roll_no, class_name, marks):

    conn = connect()

    cur = conn.cursor()

    cur.execute("INSERT INTO students (name, roll_no,
class_name, marks) VALUES (%s, %s, %s, %s)",

                (name, roll_no, class_name, marks))

    conn.commit()

    conn.close()

# View all students

def view_students():

    conn = connect()

    cur = conn.cursor()
```

```
cur.execute("SELECT * FROM students")
```

```
rows = cur.fetchall()
```

```
conn.close()
```

```
return rows
```

```
# Search students by name or roll number
```

```
def search_student(keyword):
```

```
    conn = connect()
```

```
    cur = conn.cursor()
```

```
    cur.execute("SELECT * FROM students WHERE name  
LIKE %s OR roll_no LIKE %s",
```

```
                (f"%{keyword}%", f"%{keyword}%"))
```

```
    rows = cur.fetchall()
```

```
    conn.close()
```

```
    return rows
```

```
# Update student marks
```

```
def update_marks(student_id, marks):
```

```
conn = connect()
```

```
cur = conn.cursor()
```

```
cur.execute("UPDATE students SET marks = %s WHERE id  
= %s", (marks, student_id))
```

```
conn.commit()
```

```
conn.close()
```

```
# Delete a student
```

```
def delete_student(student_id):
```

```
    conn = connect()
```

```
    cur = conn.cursor()
```

```
    cur.execute("DELETE FROM students WHERE id = %s",  
(student_id,))
```

```
    conn.commit()
```

```
    conn.close()
```

```
# Create table if not exists
```

```
def create_table():
```

```
    conn = connect()
```

15

```
cur = conn.cursor()
```

```
cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS students (
```

```
    id INT PRIMARY KEY AUTO_INCREMENT,
```

```
    name VARCHAR(255) NOT NULL,
```

```
    roll_no VARCHAR(50) UNIQUE NOT NULL,
```

```
    class_name VARCHAR(100),
```

```
    marks INT) """)
```

```
conn.commit()
```

```
conn.close()
```

```
# Menu for the system
```

```
def menu():
```

```
    create_table()
```

```
    while True:
```

```
        print("\n STUDENT MANAGEMENT SYSTEM ")
```

```
        print("1. Add Student")
```

```
print("2. View Students")

print("3. Search Student")

print("4. Update Marks")

print("5. Delete Student")

print("6. Exit")

choice = input("Enter your choice (1-6): ")

if choice == '1':

    name = input("Enter Name: ")

    roll_no = input("Enter Roll No: ")

    class_name = input("Enter Class: ")

    marks = int(input("Enter Marks: "))

    add_student(name, roll_no, class_name, marks)

    print("Student added successfully!")

elif choice == '2':

    students = view_students()

    for student in students:
```

```
print(student)
```

```
elif choice == '3':
```

```
keyword = input("Enter Name/Roll No to search: ")
```

```
results = search_student(keyword)
```

```
for student in results:
```

```
    print(student)
```

```
elif choice == '4':
```

```
    student_id = int(input("Enter Student ID to update  
marks: "))
```

```
    marks = int(input("Enter New Marks: "))
```

```
    update_marks(student_id, marks)
```

```
    print("Marks updated successfully!")
```

```
elif choice == '5':
```

```
    student_id = int(input("Enter Student ID to delete: "))
```

```
    print("Student deleted successfully!")
```

```
elif choice == '6':
```

```
    print("Thank you for using the Student Management
```

```
System!")
```

```
    break
```

```
else:
```

```
    print("Invalid choice. Try again.")
```

```
# Run the menu
```

```
menu()
```

SCREENSHOTS:

STUDENT MANAGEMENT SYSTEM

1. Add Student
2. View Students
3. Search Student
4. Update Marks
5. Delete Student
6. Exit

Enter your choice (1-6): 3

Enter Name/Roll No to search: Jane

(2, 'Jane Smith', 'R102', '10B', 90)

STUDENT MANAGEMENT SYSTEM

1. Add Student
2. View Students
3. Search Student
4. Update Marks
5. Delete Student
6. Exit

Enter your choice (1-6): 1

Enter Name: John Doe

Enter Roll No: R101

Enter Class: 10A

Enter Marks: 85

Student added successfully!

```
STUDENT MANAGEMENT SYSTEM
1. Add Student
2. View Students
3. Search Student
4. Update Marks
5. Delete Student
6. Exit
Enter your choice (1-6): 4
Enter Student ID to update marks: 1
Enter New Marks: 95
Marks updated successfully!
```

```
Enter your choice (1-6): 2
(1, 'John Doe', 'R101', '10A', 95)
(2, 'Jane Smith', 'R102', '10B', 90)
```

```
STUDENT MANAGEMENT SYSTEM
1. Add Student
2. View Students
3. Search Student
4. Update Marks
5. Delete Student
6. Exit
Enter your choice (1-6): 5
Enter Student ID to delete: 2
Student deleted successfully!
```

```
STUDENT MANAGEMENT SYSTEM
1. Add Student
2. View Students
3. Search Student
4. Update Marks
5. Delete Student
6. Exit
Enter your choice (1-6): 3
Enter Name/Roll No to search: Jane
(2, 'Jane Smith', 'R102', '10B', 90)
```

BIBLIOGRAPHY

1. CS With Python - Class XII By: Preeti Arora
2. Computer science With Python - Class XII
By: Sumita Arora
3. STUDENT SUPPORT MATERIAL Class-XII
COMPUTER SCIENCE Session: 2024-25
4. Website: <https://www.w3resource.com>
5. Blog12ip.blogspot.com